JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Field

java.lang.Object
    java.lang.reflect.AccessibleObject
        java.lang.reflect.Field

All Implemented Interfaces:

AnnotatedElement, Member

```
public final class Field
extends AccessibleObject
implements Member
```

A `Field` provides information about, and dynamic access to, a single field of a class or an interface. The reflected field may be a class (static) field or an instance field.

A `Field` permits widening conversions to occur during a get or set access operation, but throws an `IllegalArgumentException` if a narrowing conversion would occur.

Since:

1.1

See Also:

Member, Class, Class.getFields(), Class.getField(String), Class.getDeclaredFields(), Class.getDeclaredField(String)

Field Summary

Fields declared in interface Member
DECLARED, PUBLIC

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
Set<AccessFlag>	**accessFlags**()	Returns an unmodifiable set of the access flags for this field, possibly empty.
boolean	**equals**(Object obj)	Compares this `Field` against the specified object.

Object	**get**(Object obj)	Returns the value of the field represented by this Field, on the specified object.
AnnotatedType	**getAnnotatedType**()	Returns an AnnotatedType object that represents the use of a type to specify the declared type of the field represented by this Field.
<T extends Annotation> T	**getAnnotation**(Class<T> annotationClas	Returns this element's annotation for the specified type if such an annotation is *present*, else null.
<T extends Annotation> T[]	**getAnnotationsByType**(Class<T> annotationClas	Returns annotations that are *associated* with this element.
boolean	**getBoolean**(Object obj)	Gets the value of a static or instance boolean field.
byte	**getByte**(Object obj)	Gets the value of a static or instance byte field.
char	**getChar**(Object obj)	Gets the value of a static or instance field of type char or of another primitive type convertible to type char via a widening conversion.
Class<?>	**getDeclaringClass**()	Returns the Class object representing the class or interface that declares the field represented by this Field object.
double	**getDouble**(Object obj)	Gets the value of a static or instance field of type

		double or of another primitive type convertible to type double via a widening conversion.
float	**getFloat**(Object obj)	Gets the value of a static or instance field of type float or of another primitive type convertible to type float via a widening conversion.
Type	**getGenericType**()	Returns a Type object that represents the declared type for the field represented by this Field object.
int	**getInt**(Object obj)	Gets the value of a static or instance field of type int or of another primitive type convertible to type int via a widening conversion.
long	**getLong**(Object obj)	Gets the value of a static or instance field of type long or of another primitive type convertible to type long via a widening conversion.
int	**getModifiers**()	Returns the Java language modifiers for the field represented by this Field object, as an integer.
String	**getName**()	

		Returns the name of the field represented by this `Field` object.
`short`	**`getShort`**`(Object obj)`	Gets the value of a static or instance field of type `short` or of another primitive type convertible to type `short` via a widening conversion.
`Class<?>`	**`getType`**`()`	Returns a `Class` object that identifies the declared type for the field represented by this `Field` object.
`int`	**`hashCode`**`()`	Returns a hashcode for this `Field`.
`boolean`	**`isEnumConstant`**`()`	Returns `true` if this field represents an element of an enumerated class; returns `false` otherwise.
`boolean`	**`isSynthetic`**`()`	Returns `true` if this field is a synthetic field; returns `false` otherwise.
`void`	**`set`**`(Object obj, Object value)`	Sets the field represented by this `Field` object on the specified object argument to the specified new value.
`void`	**`setAccessible`** `(boolean flag)`	Set the `accessible` flag for this reflected object to the indicated boolean value.
`void`	**`setBoolean`**`(Object obj, boolean z)`	Sets the value of a field as a `boolean` on the specified object.

void	**setByte**(Object obj, byte b)	Sets the value of a field as a byte on the specified object.
void	**setChar**(Object obj, char c)	Sets the value of a field as a char on the specified object.
void	**setDouble**(Object obj, double d)	Sets the value of a field as a double on the specified object.
void	**setFloat**(Object obj, float f)	Sets the value of a field as a float on the specified object.
void	**setInt**(Object obj, int i)	Sets the value of a field as an int on the specified object.
void	**setLong**(Object obj, long l)	Sets the value of a field as a long on the specified object.
void	**setShort**(Object obj, short s)	Sets the value of a field as a short on the specified object.
String	**toGenericString**()	Returns a string describing this Field, including its generic type.
String	**toString**()	Returns a string describing this Field.

Methods declared in class AccessibleObject

canAccess, getAnnotations, getDeclaredAnnotation, getDeclaredAnnotations, getDeclaredAnnotationsByType, isAccessible, isAnnotationPresent, setAccessible, trySetAccessible

Methods declared in class Object

`clone`, `finalize`, `getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Method Details

setAccessible

```
public void setAccessible(boolean flag)
```

Set the `accessible` flag for this reflected object to the indicated boolean value. A value of `true` indicates that the reflected object should suppress checks for Java language access control when it is used. A value of `false` indicates that the reflected object should enforce checks for Java language access control when it is used, with the variation noted in the class description.

This method may be used by a caller in class `C` to enable access to a `member` of `declaring class` `D` if any of the following hold:

- `C` and `D` are in the same module.
- The member is `public` and `D` is `public` in a package that the module containing `D` `exports` to at least the module containing `C`.
- The member is `protected static`, `D` is `public` in a package that the module containing `D` exports to at least the module containing `C`, and `C` is a subclass of `D`.
- `D` is in a package that the module containing `D` `opens` to at least the module containing `C`. All packages in unnamed and open modules are open to all modules and so this method always succeeds when `D` is in an unnamed or open module.

This method may be used by JNI code with no caller class on the stack to enable access to a `member` of `declaring class` `D` if and only if:

- The member is `public` and `D` is `public` in a package that the module containing `D` `exports` unconditionally.

This method cannot be used to enable access to private members, members with default (package) access, protected instance members, or protected constructors when the declaring class is in a different module to the caller and the package containing the declaring class is not open to the caller's module.

If this reflected object represents a non-final field, and this method is used to enable access, then both *read* and *write* access to the field are enabled.

If this reflected object represents a *non-modifiable* final field then enabling access only enables read access. Any attempt to set the field value throws an `IllegalAccessException`. The following fields are non-modifiable:

- static final fields declared in any class or interface
- final fields declared in a record
- final fields declared in a hidden class

If this reflected object represents a non-static final field in a class that is not a record class or hidden class, then enabling access will enable read access. Whether write access is allowed or not is checked when attempting to set the field value.

Overrides:

`setAccessible` in class `AccessibleObject`

Parameters:

`flag` - the new value for the `accessible` flag

Throws:

`InaccessibleObjectException` - if access cannot be enabled

External Specifications

Java Native Interface Specification

See Also:

`AccessibleObject.trySetAccessible()`,
`MethodHandles.privateLookupIn(Class, MethodHandles.Lookup)`

getDeclaringClass

```
public Class<?> getDeclaringClass()
```

Returns the `Class` object representing the class or interface that declares the field represented by this `Field` object.

Specified by:

`getDeclaringClass` in interface `Member`

Returns:

an object representing the declaring class of the underlying member

getName

```
public String getName()
```

Returns the name of the field represented by this `Field` object.

Specified by:

`getName` in interface `Member`

Returns:

the simple name of the underlying member

getModifiers

```
public int getModifiers()
```

Returns the Java language modifiers for the field represented by this `Field` object, as an integer. The `Modifier` class should be used to decode the modifiers.

Specified by:

`getModifiers` in interface `Member`

Returns:

the Java language modifiers for the underlying member

See *Java Language Specification*:

8.3 Field Declarations
9.3 Field (Constant) Declarations

See Also:

`Modifier`, `accessFlags()`

accessFlags

```
public Set<AccessFlag> accessFlags()
```

Returns an unmodifiable set of the access flags for this field, possibly empty.

Specified by:

`accessFlags` in interface `Member`

Returns:

an unmodifiable set of the access flags for this field, possibly empty

See *Java Virtual Machine Specification*:

4.5 Fields

Since:

20

See Also:

`getModifiers()`

isEnumConstant

```
public boolean isEnumConstant()
```

Returns `true` if this field represents an element of an enumerated class; returns `false` otherwise.

Returns:

`true` if and only if this field represents an element of an enumerated class.

See *Java Language Specification*:

8.9.1 Enum Constants

Since:

1.5

isSynthetic

```
public boolean isSynthetic()
```

Returns `true` if this field is a synthetic field; returns `false` otherwise.

Specified by:

`isSynthetic` in interface `Member`

Returns:

true if and only if this field is a synthetic field as defined by the Java Language Specification.

Since:

1.5

See Also:

Java programming language and JVM modeling in core reflection

getType

```
public Class<?> getType()
```

Returns a `Class` object that identifies the declared type for the field represented by this `Field` object.

Returns:

a `Class` object identifying the declared type of the field represented by this object

getGenericType

```
public Type getGenericType()
```

Returns a `Type` object that represents the declared type for the field represented by this `Field` object.

If the declared type of the field is a parameterized type, the `Type` object returned must accurately reflect the actual type arguments used in the source code.

If the type of the underlying field is a type variable or a parameterized type, it is created. Otherwise, it is resolved.

Returns:

a `Type` object that represents the declared type for the field represented by this `Field` object

Throws:

`GenericSignatureFormatError` - if the generic field signature does not conform to the format specified in *The Java Virtual Machine Specification*

`TypeNotPresentException` - if the generic type signature of the underlying field refers to a non-existent class or interface declaration

`MalformedParameterizedTypeException` - if the generic signature of the underlying field refers to a parameterized type that cannot be instantiated for any reason

Since:

1.5

equals

```
public boolean equals(Object obj)
```

Compares this `Field` against the specified object. Returns true if the objects are the same. Two `Field` objects are the same if they were declared by the same class and have the same name and type.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the reference object with which to compare.

Returns:

`true` if this object is the same as the obj argument; `false` otherwise.

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

Returns a hashcode for this `Field`. This is computed as the exclusive-or of the hashcodes for the underlying field's declaring class name and its name.

Overrides:

`hashCode` in class `Object`

Returns:

a hash code value for this object

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
public String toString()
```

Returns a string describing this `Field`. The format is the access modifiers for the field, if any, followed by the field type, followed by a space, followed by the fully-qualified name of the class declaring the field, followed by a period, followed by the name of the field. For example:

```
public static final int java.lang.Thread.MIN_PRIORITY
private int java.io.FileDescriptor.fd
```

The modifiers are placed in canonical order as specified by "The Java Language Specification". This is `public`, `protected` or `private` first, and then other modifiers in the following order: `static`, `final`, `transient`, `volatile`.

Overrides:

`toString` in class `Object`

Returns:

a string describing this `Field`

See *Java Language Specification*:

8.3.1 Field Modifiers

toGenericString

```
public String toGenericString()
```

Returns a string describing this `Field`, including its generic type. The format is the access modifiers for the field, if any, followed by the generic field type, followed by a space, followed by the fully-qualified name of the class declaring the field, followed by a period, followed by the name of the field.

The modifiers are placed in canonical order as specified by "The Java Language Specification". This is `public`, `protected` or `private` first, and then other modifiers in the following order: `static`, `final`, `transient`, `volatile`.

Returns:

a string describing this `Field`, including its generic type

See *Java Language Specification*:

8.3.1 Field Modifiers

Since:

1.5

get

```
public Object get(Object obj)
           throws IllegalArgumentException,
                  IllegalAccessException
```

Returns the value of the field represented by this `Field`, on the specified object. The value is automatically wrapped in an object if it has a primitive type.

The underlying field's value is obtained as follows:

If the underlying field is a static field, the `obj` argument is ignored; it may be null.

Otherwise, the underlying field is an instance field. If the specified `obj` argument is null, the method throws a `NullPointerException`. If the specified object is not an instance of the class or interface declaring the underlying field, the method throws an `IllegalArgumentException`.

If this `Field` object is enforcing Java language access control, and the underlying field is inaccessible, the method throws an `IllegalAccessException`. If the underlying field is static, the class that declared the field is initialized if it has not already been initialized.

Otherwise, the value is retrieved from the underlying instance or static field. If the field has a primitive type, the value is wrapped in an object before being returned, otherwise it is returned as is.

If the field is hidden in the type of `obj`, the field's value is obtained according to the preceding rules.

Parameters:

`obj` - object from which the represented field's value is to be extracted

Returns:

the value of the represented field in object `obj`; primitive values are wrapped in an appropriate object before being returned

Throws:

`IllegalAccessException` - if this `Field` object is enforcing Java language access control and the underlying field is inaccessible.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof).

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

getBoolean

```
public boolean getBoolean(Object obj)
                   throws IllegalArgumentException,
                          IllegalAccessException
```

Gets the value of a static or instance `boolean` field.

Parameters:

`obj` - the object to extract the `boolean` value from

Returns:

the value of the `boolean` field

Throws:

IllegalAccessException - if this `Field` object is enforcing Java language access control and the underlying field is inaccessible.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type `boolean` by a widening conversion.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

get(Object)

getByte

```
public byte getByte(Object obj)
             throws IllegalArgumentException,
                    IllegalAccessException
```

Gets the value of a static or instance `byte` field.

Parameters:

`obj` - the object to extract the `byte` value from

Returns:

the value of the `byte` field

Throws:

`IllegalAccessException` - if this `Field` object is enforcing Java language access control and the underlying field is inaccessible.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type `byte` by a widening conversion.

`NullPointerException` - if the specified object is null and the field is an instance field.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

See Also:

`get(Object)`

getChar

```
public char getChar(Object obj)
             throws IllegalArgumentException,
                    IllegalAccessException
```

Gets the value of a static or instance field of type `char` or of another primitive type convertible to type `char` via a widening conversion.

Parameters:

`obj` - the object to extract the `char` value from

Returns:

the value of the field converted to type `char`

Throws:

`IllegalAccessException` - if this `Field` object is enforcing Java language access control and the underlying field is inaccessible.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type `char` by a widening conversion.

`NullPointerException` - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

get(Object)

getShort

```
public short getShort(Object obj)
               throws IllegalArgumentException,
                      IllegalAccessException
```

Gets the value of a static or instance field of type short or of another primitive type convertible to type short via a widening conversion.

Parameters:

obj - the object to extract the short value from

Returns:

the value of the field converted to type short

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is inaccessible.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type short by a widening conversion.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

get(Object)

getInt

```
public int getInt(Object obj)
           throws IllegalArgumentException,
                  IllegalAccessException
```

Gets the value of a static or instance field of type int or of another primitive type convertible to type int via a widening conversion.

Parameters:

obj - the object to extract the int value from

Returns:

the value of the field converted to type `int`

Throws:

`IllegalAccessException` - if this `Field` object is enforcing Java language access control and the underlying field is inaccessible.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type `int` by a widening conversion.

`NullPointerException` - if the specified object is null and the field is an instance field.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

See Also:

`get(Object)`

getLong

```
public long getLong(Object obj)
             throws IllegalArgumentException,
                    IllegalAccessException
```

Gets the value of a static or instance field of type `long` or of another primitive type convertible to type `long` via a widening conversion.

Parameters:

`obj` - the object to extract the `long` value from

Returns:

the value of the field converted to type `long`

Throws:

`IllegalAccessException` - if this `Field` object is enforcing Java language access control and the underlying field is inaccessible.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type `long` by a widening conversion.

`NullPointerException` - if the specified object is null and the field is an instance field.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

See Also:

get(Object)

getFloat

```
public float getFloat(Object obj)
               throws IllegalArgumentException,
                      IllegalAccessException
```

Gets the value of a static or instance field of type `float` or of another primitive type convertible to type `float` via a widening conversion.

Parameters:

`obj` - the object to extract the `float` value from

Returns:

the value of the field converted to type `float`

Throws:

`IllegalAccessException` - if this `Field` object is enforcing Java language access control and the underlying field is inaccessible.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type `float` by a widening conversion.

`NullPointerException` - if the specified object is null and the field is an instance field.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

See Also:

get(Object)

getDouble

```
public double getDouble(Object obj)
                 throws IllegalArgumentException,
                        IllegalAccessException
```

Gets the value of a static or instance field of type `double` or of another primitive type convertible to type `double` via a widening conversion.

Parameters:

`obj` - the object to extract the `double` value from

Returns:

the value of the field converted to type `double`

Throws:

`IllegalAccessException` - if this `Field` object is enforcing Java language access control and the underlying field is inaccessible.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type `double` by a widening conversion.

`NullPointerException` - if the specified object is null and the field is an instance field.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

See Also:

`get(Object)`

set

```
public void set(Object obj,
                Object value)
         throws IllegalArgumentException,
                IllegalAccessException
```

Sets the field represented by this `Field` object on the specified object argument to the specified new value. The new value is automatically unwrapped if the underlying field has a primitive type.

The operation proceeds as follows:

If the underlying field is static, the `obj` argument is ignored; it may be null.

Otherwise the underlying field is an instance field. If the specified object argument is null, the method throws a `NullPointerException`. If the specified object argument is not an instance of the class or interface declaring the underlying field, the method throws an `IllegalArgumentException`.

If this `Field` object is enforcing Java language access control, and the underlying field is inaccessible, the method throws an `IllegalAccessException`.

If the underlying field is final, this `Field` object has *write* access if and only if all of the following conditions are true, where `D` is the field's declaring class:

- `setAccessible(true)` has succeeded for this `Field` object.
- final field mutation is enabled for the caller's module.
- At least one of the following conditions holds:
    1. `D` and the caller class are in the same module.

    2. The field is `public` and `D` is `public` in a package that the module containing `D` exports to at least the caller's module.
    3. `D` is in a package that is open to the caller's module.
- `D` is not a record class.
- `D` is not a hidden class.
- The field is non-static.

If any of the above conditions is not met, this method throws an `IllegalAccessException`.

These conditions are more restrictive than the conditions specified by `setAccessible(boolean)` to suppress access checks. In particular, updating a module to export or open a package cannot be used to allow *write* access to final fields with the `set` methods defined by `Field`. Condition (b) is not met if the module containing `D` has been updated with addExports to export the package to the caller's module. Condition (c) is not met if the module containing `D` has been updated with addOpens to open the package to the caller's module.

This method may be called by JNI code with no caller class on the stack. In that case, and when the underlying field is final, this `Field` object has *write* access if and only if all of the following conditions are true, where `D` is the field's declaring class:

- `setAccessible(true)` has succeeded for this `Field` object.
- final field mutation is enabled for the unnamed module.
- The field is `public` and `D` is `public` in a package that is exported to all modules.
- `D` is not a record class.
- `D` is not a hidden class.
- The field is non-static.

If any of the above conditions is not met, this method throws an `IllegalAccessException`.

Setting a final field in this way is meaningful only during deserialization or reconstruction of instances of classes with blank final fields, before they are made available for access by other parts of a program. Use in any other context may have unpredictable effects, including cases in which other parts of a program continue to use the original value of this field.

If the underlying field is of a primitive type, an unwrapping conversion is attempted to convert the new value to a value of a primitive type. If this attempt fails, the method throws an `IllegalArgumentException`.

If, after possible unwrapping, the new value cannot be converted to the type of the underlying field by an identity or widening conversion, the method throws an `IllegalArgumentException`.

If the underlying field is static, the class that declared the field is initialized if it has not already been initialized.

The field is set to the possibly unwrapped and widened new value.

If the field is hidden in the type of `obj`, the field's value is set according to the preceding rules.

Parameters:

`obj` - the object whose field should be modified

`value` - the new value for the field of `obj` being modified

Throws:

`IllegalAccessException` - if this `Field` object is enforcing Java language access control and the underlying field is inaccessible or final; or if this `Field` object has no write access.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

`NullPointerException` - if the specified object is null and the field is an instance field.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

See Also:

Mutation methods

setBoolean

```
public void setBoolean(Object obj,
                       boolean z)
                throws IllegalArgumentException,
                       IllegalAccessException
```

Sets the value of a field as a `boolean` on the specified object. This method is equivalent to `set(obj, zObj)`, where `zObj` is a `Boolean` object and `zObj.booleanValue() == z`.

Parameters:

`obj` - the object whose field should be modified

`z` - the new value for the field of `obj` being modified

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final; or if this Field object has no write access.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

set(Object, Object)

setByte

```
public void setByte(Object obj,
                    byte b)
             throws IllegalArgumentException,
                    IllegalAccessException
```

Sets the value of a field as a byte on the specified object. This method is equivalent to set(obj, bObj), where bObj is a Byte object and bObj.byteValue() == b.

Parameters:

obj - the object whose field should be modified

b - the new value for the field of obj being modified

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final; or if this Field object has no write access.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

set(Object, Object)

setChar

```
public void setChar(Object obj,
                    char c)
             throws IllegalArgumentException,
                    IllegalAccessException
```

Sets the value of a field as a `char` on the specified object. This method is equivalent to `set(obj, cObj)`, where `cObj` is a `Character` object and `cObj.charValue() == c`.

Parameters:

`obj` - the object whose field should be modified

`c` - the new value for the field of `obj` being modified

Throws:

`IllegalAccessException` - if this `Field` object is enforcing Java language access control and the underlying field is either inaccessible or final; or if this `Field` object has no write access.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

`NullPointerException` - if the specified object is null and the field is an instance field.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

See Also:

`set(Object, Object)`

setShort

```
public void setShort(Object obj,
                     short s)
              throws IllegalArgumentException,
                     IllegalAccessException
```

Sets the value of a field as a `short` on the specified object. This method is equivalent to `set(obj, sObj)`, where `sObj` is a `Short` object and `sObj.shortValue() == s`.

Parameters:

`obj` - the object whose field should be modified

`s` - the new value for the field of `obj` being modified

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final; or if this Field object has no write access.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

set(Object, Object)

setInt

```
public void setInt(Object obj,
                   int i)
            throws IllegalArgumentException,
                   IllegalAccessException
```

Sets the value of a field as an int on the specified object. This method is equivalent to set(obj, iObj), where iObj is an Integer object and iObj.intValue() == i.

Parameters:

obj - the object whose field should be modified

i - the new value for the field of obj being modified

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final; or if this Field object has no write access.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

set(Object, Object)

setLong

```
public void setLong(Object obj,
                    long l)
             throws IllegalArgumentException,
                    IllegalAccessException
```

Sets the value of a field as a `long` on the specified object. This method is equivalent to `set(obj, lObj)`, where `lObj` is a `Long` object and `lObj.longValue() == l`.

Parameters:

`obj` - the object whose field should be modified

`l` - the new value for the field of `obj` being modified

Throws:

`IllegalAccessException` - if this `Field` object is enforcing Java language access control and the underlying field is either inaccessible or final; or if this `Field` object has no write access.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

`NullPointerException` - if the specified object is null and the field is an instance field.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

See Also:

`set(Object, Object)`

setFloat

```
public void setFloat(Object obj,
                     float f)
              throws IllegalArgumentException,
                     IllegalAccessException
```

Sets the value of a field as a `float` on the specified object. This method is equivalent to `set(obj, fObj)`, where `fObj` is a `Float` object and `fObj.floatValue() == f`.

Parameters:

`obj` - the object whose field should be modified

`f` - the new value for the field of `obj` being modified

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final; or if this Field object has no write access.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

set(Object, Object)

setDouble

```
public void setDouble(Object obj,
                      double d)
               throws IllegalArgumentException,
                      IllegalAccessException
```

Sets the value of a field as a double on the specified object. This method is equivalent to set(obj, dObj), where dObj is a Double object and dObj.doubleValue() == d.

Parameters:

obj - the object whose field should be modified

d - the new value for the field of obj being modified

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final; or if this Field object has no write access.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

set(Object, Object)

getAnnotation

```
public <T extends Annotation> T getAnnotation
(Class<T> annotationClass)
```

Returns this element's annotation for the specified type if such an annotation is *present*, else null.

Note that any annotation returned by this method is a declaration annotation.

Specified by:

`getAnnotation` in interface `AnnotatedElement`

Overrides:

`getAnnotation` in class `AccessibleObject`

Type Parameters:

`T` - the type of the annotation to query for and return if present

Parameters:

`annotationClass` - the Class object corresponding to the annotation type

Returns:

this element's annotation for the specified annotation type if present on this element, else null

Throws:

`NullPointerException` - if the given annotation class is null

Since:

1.5

getAnnotationsByType

```
public <T extends Annotation> T[] getAnnotationsByType
(Class<T> annotationClass)
```

Returns annotations that are *associated* with this element. If there are no annotations *associated* with this element, the return value is an array of length 0. The difference between this method and `AnnotatedElement.getAnnotation(Class)` is that this method detects if its argument is a *repeatable annotation type* (JLS §9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Note that any annotations returned by this method are declaration annotations.

Specified by:

getAnnotationsByType in interface AnnotatedElement

Overrides:

getAnnotationsByType in class AccessibleObject

Type Parameters:

T - the type of the annotation to query for and return if present

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

all this element's annotations for the specified annotation type if associated with this element, else an array of length zero

Throws:

NullPointerException - if the given annotation class is null

Since:

1.8

getAnnotatedType

```
public AnnotatedType getAnnotatedType()
```

Returns an AnnotatedType object that represents the use of a type to specify the declared type of the field represented by this Field.

Returns:

an object representing the declared type of the field represented by this Field

Since:

1.8

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.